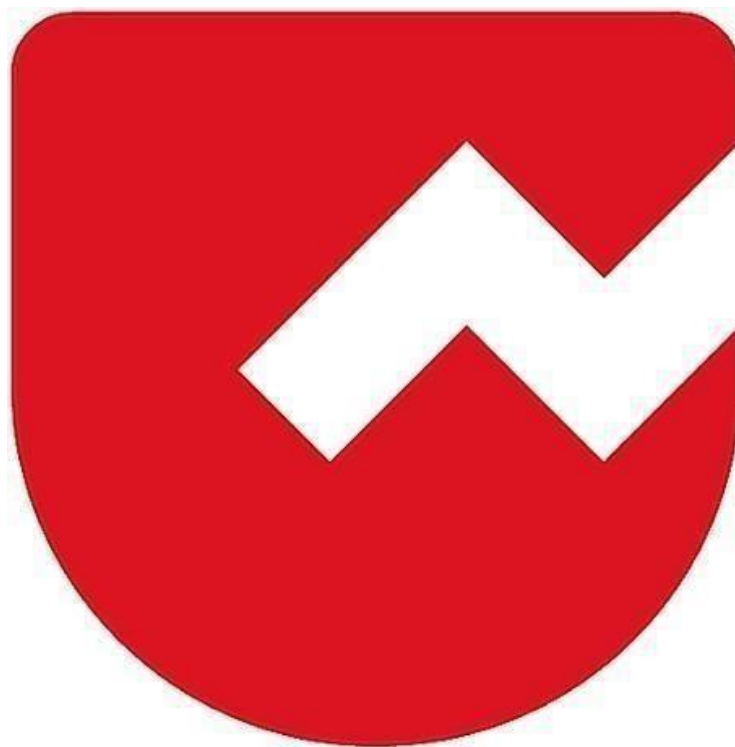




Zoho Schools for Graduate Studies



Notes

Multithreading

Multithreading is inspired from multitasking of humans.

Multitasking

- Process based
 - two or more programs running concurrently
 - e.g., Typing in an editor or browsing and hearing songs
- Thread based
 - Within a program, tasks run concurrently
 - e.g., Navigating through a web page in one tab and downloading a file from another tab in a web browser

Two ways of implementation in JAVA

1. By extending Thread class
2. By implementing Runnable interface

By extending Thread class

```
class MyFirstThread extends Thread {  
    public void run() {  
        for(int i = 0; i < 5; i++) {  
            System.out.println("Vanakkam ZOHO");  
        }  
    }  
}
```

By extending Thread class

```
public class MyThreadProgram {  
    public static void main(String args[]) {  
        MyFirstThread threadObject = new MyFirstThread();  
        threadObject.start();  
    }  
}
```

By extending Thread class

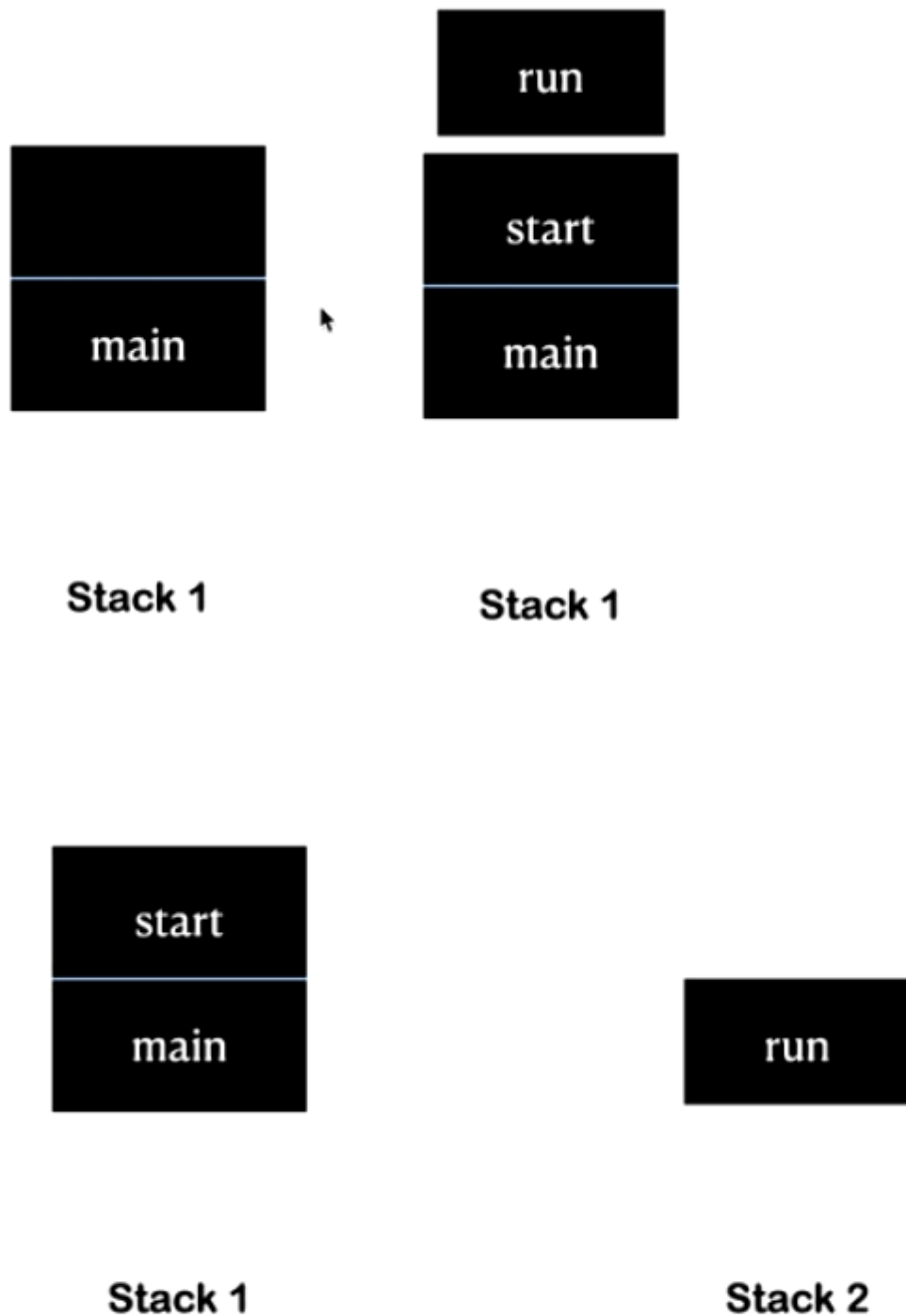
```
public class MyThreadProgram {  
    public static void main(String args[]) {  
        MyFirstThread threadObject = new MyFirstThread();  
        threadObject.start();  
threadObject.run();  
    }  
}
```

Things to remember

1. Runtime polymorphism

- `run()` is overridden from Thread class
- `start()` method is defined in Thread class

2. Function call stack



Method - 2

By implementing Runnable interface

```
public class MyRunnableImplementation implements Runnable {  
  
    public void run(){  
        System.out.println("Vanakkam from a runnable thread!");  
    }  
  
}
```

By implementing Runnable interface

```
public class MyRunnableThreadProgram {  
  
    public static void main(String args[]) {  
        Runnable runnableObject = new MyRunnableImplementation();  
    }  
  
}
```

By implementing Runnable interface

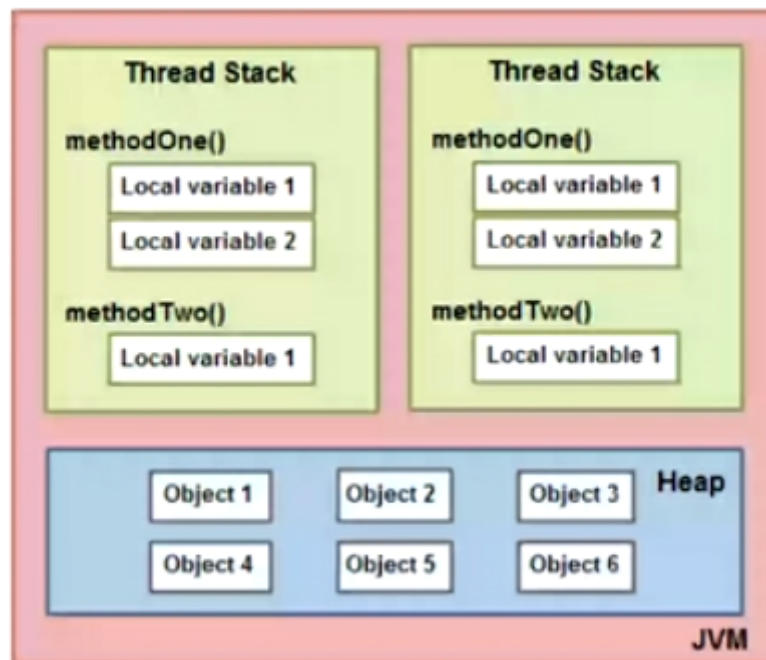
```
public class MyRunnableThreadProgram {  
  
    public static void main(String args[]) {  
        Runnable runnableObject = new MyRunnableImplementation();  
        runnableObject.start();  
        runnableObject.run();  
    }  
}
```

By implementing Runnable interface

```
public class MyRunnableThreadProgram {  
  
    public static void main(String args[]) {  
        Runnable runnableObject = new MyRunnableImplementation();  
        Thread threadObject = new Thread(runnableObject);  
        threadObject.start();  
    }  
}
```

Constructor used: Thread(Runnable rObject)
--

Java Memory Model



Heap - dynamically allocated - OutOfMemoryException

Threads:

- A **thread** is a lightweight sub-process that represents an independent path of execution within a program.
- In Java, every program has **at least one thread**, called the **main thread**, which starts automatically when the program begins execution.
- Threads allow multiple tasks to run **concurrently**, improving performance and responsiveness.

Types of Threads:

Main Thread (Default Thread)

- Created automatically when the Java program starts.
- Responsible for executing the main() method.

Child Threads (User-defined threads)

- Created by the programmer to run tasks parallelly with the main thread.

Defining a Thread in Java:

A thread is defined by overriding the run () method.

run() Method

- Contains the logic that the thread will execute.
- When a thread starts, only the run () method code runs.

How to create a Thread in Java:

There are two ways:

1. Extending the Thread class
2. Implementing the Runnable interface

Method 1 – Extending the Thread Class:

Steps

1. Create a class that extends Thread.
2. Override the run() method.
3. Create an object of your class.
4. Call the start() method → this internally calls run().

Important Notes

- You must use **start()** (NOT run()) to start a new thread.
- A thread **executes only once**. Restarting a thread causes:
→ **IllegalThreadStateException**.

Method 2 – Implementing Runnable Interface:

Why Runnable is better?

- Java supports multiple inheritance only through interfaces.
- To extend another class and still create a thread → Runnable is recommended.
- More flexible for large applications.

Steps

1. Create a class that implements Runnable.
2. Override run().
3. Create a Thread object and pass your Runnable object to it.
4. Call Start().